2026-10-26 MySQL索引：下

- 1. 从页目录到 B+ 树：索引“下”部分
  - 1.1 承接上部分
    - 1.1.1 从“页内目录”继续向上
    - 1.1.2 索引的本质是数据结构
    - 1.1.3 本部分知识地图
- 2. 为什么选择 B+ 树
  - 2.1 B 树与 B+ 树的区别
    - 2.1.1 磁盘索引的首要目标是减少 Page 访问
    - 2.1.2 B 树：内部节点也保存数据
    - 2.1.3 B+ 树：数据集中在叶节点
    - 2.1.4 对本节最重要的两点差异
  - 2.2 为什么其他结构不适合作为 InnoDB 的主索引
    - 2.2.1 链表、二叉搜索树和 AVL/红黑树
    - 2.2.2 哈希索引
    - 2.2.3 InnoDB 与 MyISAM 为什么采用 B+ 树
  - 2.3 从页目录理解树高
    - 2.3.1 每一层目录都在缩小范围
    - 2.3.2 根到叶的路径就是索引查找路径
- 3. InnoDB 聚簇索引
  - 3.1 索引与数据保存在同一棵主键树
    - 3.1.1 聚簇索引的含义
    - 3.1.2 为什么主键设计很重要
    - 3.1.3 从表文件观察聚簇索引
  - 3.2 InnoDB 辅助索引与回表
    - 3.2.1 辅助索引叶节点保存主键
    - 3.2.2 回表查询的两步路径
    - 3.2.3 覆盖索引为什么能减少回表
  - 3.3 一张表为什么对应多棵树
    - 3.3.1 主键树和辅助索引树同时存在
    - 3.3.2 更新辅助索引列的代价
- 4. MyISAM 非聚簇索引
  - 4.1 数据与索引分文件保存
    - 4.1.1 MyISAM 的典型文件
    - 4.1.2 为什么称为非聚簇索引
  - 4.2 MyISAM 主键索引

        - 4.2.1 叶节点保存行地址
        - 4.2.2 从索引到数据通常多一步寻址
    - 4.3 MyISAM 辅助索引
        - 4.3.1 与主键索引结构相同
        - 4.3.2 InnoDB 与 MyISAM 对照
- 5. 索引操作
    - 5.1 创建主键索引
        - 5.1.1 在字段定义处声明
        - 5.1.2 在表定义末尾声明
        - 5.1.3 建表后再添加
    - 5.2 创建唯一索引
        - 5.2.1 三种常见写法
        - 5.2.2 唯一索引的特点
    - 5.3 创建普通索引
        - 5.3.1 建表时声明
        - 5.3.2 建表后添加
        - 5.3.3 显式指定索引名
    - 5.4 创建全文索引
        - 5.4.1 示例表
        - 5.4.2 普通 LIKE 不一定使用全文索引
        - 5.4.3 使用 MATCH ... AGAINST
- 6. 查看、删除与设计索引
    - 6.1 查看索引
        - 6.1.1 三种常用方式
        - 6.1.2 重点字段
    - 6.2 删除索引
        - 6.2.1 删除主键
        - 6.2.2 删除普通索引或唯一索引
        - 6.2.3 主键和唯一键共存时
    - 6.3 索引创建原则
        - 6.3.1 适合建索引的场景
        - 6.3.2 不适合单独建索引的场景
        - 6.3.3 索引设计要结合查询整体判断
    - 6.4 复合索引、最左匹配与覆盖索引
        - 6.4.1 复合索引
        - 6.4.2 最左匹配原则
        - 6.4.3 覆盖索引

- 7. 易错点与修正
  - 7.1 Index_type: BTREE 不是二叉搜索树
  - 7.2 MyISAM 与 InnoDB 的“索引和数据”关系
  - 7.3 文件名与版本问题
  - 7.4 全文索引的版本差异
  - 7.5 唯一索引不等于主键
- 8. 总结与复习
  - 8.1 一张图串起索引“下”部分
  - 8.2 关键结论
    - 8.2.1 B+ 树为磁盘访问而优化
    - 8.2.2 InnoDB 主键决定聚簇索引
    - 8.2.3 InnoDB 辅助索引通常需要回表
    - 8.2.4 MyISAM 是非聚簇索引
    - 8.2.5 索引是一笔读写权衡
  - 8.3 自测题
    - 8.3.1 为什么 B+ 树比二叉搜索树更适合磁盘索引?
    - 8.3.2 B+ 树为什么比 B 树更适合范围查询?
    - 8.3.3 InnoDB 辅助索引为什么要回表?
    - 8.3.4 MyISAM 主键索引为什么不是聚簇索引?
    - 8.3.5 为什么 SHOW INDEX 比只执行 DESC 更适合检查索引?

1. 从页目录到 B+ 树：索引“下”部分

> **学习目标：**
>
> 1. 理解 B+ 树为什么比普通二叉树更适合磁盘索引。
> 2. 分清 B 树与 B+ 树在数据存放和范围查询上的差异。
> 3. 理解 InnoDB 聚簇索引、辅助索引和回表。
> 4. 理解 MyISAM 非聚簇索引的文件和数据定位方式。
> 5. 掌握主键、唯一、普通、全文索引的创建与删除。
> 6. 能通过 `SHOW INDEX`、`EXPLAIN` 判断索引是否建立和是否被使用。

1.1 承接上部分

1.1.1 从“页内目录”继续向上

上部分已经得到一条查找路径：

```
有序记录
    ↓ 页内目录
缩小页内范围
    ↓ 页目录
缩小目标 Page 范围
    ↓
目标 Page
```

当 Page 数量继续增长，单个目录页也放不下所有 Page 的键值和指针，目录本身就必须继续分层。把这一思路重复下去，就得到一棵多阶搜索树，也就是 InnoDB 使用的 B+ 树索引。

1.1.2 索引的本质是数据结构

课件强调：

> 索引的本质就是数据结构。

建立索引并不意味着复制一条条记录到内存里，而是在数据库文件中维护额外的有序树结构。索引项保存搜索键和定位信息，查询时先读少量索引 Page，再定位到目标数据。

一张 InnoDB 表可以拥有主键聚簇索引以及多个辅助索引，因此同一张表可能对应多棵 B+ 树。每个索引都是一套独立的键值组织与维护成本。

1.1.3 本部分知识地图

主题	核心问题	结论
B+ 树	为什么适合磁盘	多阶、低树高、内部页只放键和指针
B 树对比	为什么不用 B 树	B+ 树扇出更大，叶节点相连更适合范围查询
InnoDB	数据和索引怎样放	主键聚簇索引保存整行；辅助索引保存主键
MyISAM	数据和索引怎样放	`MYD` 存数据，`MYI` 存索引，叶节点保存行地址
索引操作	怎样建立和维护	主键、唯一、普通、全文四类索引
索引治理	怎样判断是否有效	`SHOW INDEX` 看结构，`EXPLAIN` 看执行计划

2. 为什么选择 B+ 树

2.1 B 树与 B+ 树的区别

2.1.1 磁盘索引的首要目标是减少 Page 访问

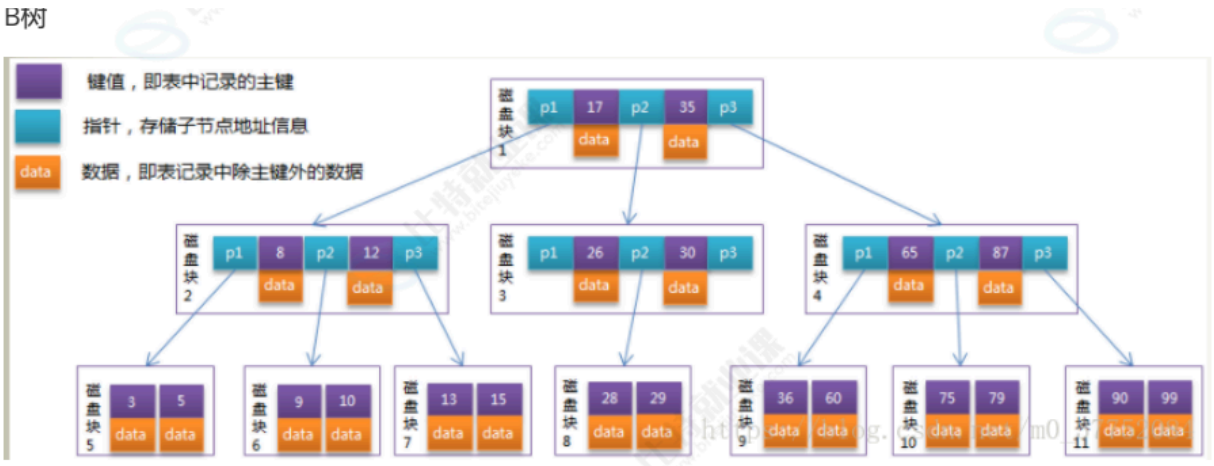
索引查询不是在内存数组中随机访问，而是需要把索引 Page 从磁盘读入 Buffer Pool。因此评价索引结构不能只看单次比较次数，还要看：

- 树有多高，因为每下降一层通常可能多读一个 Page。
- 每个节点能容纳多少键值和子指针。
- 范围查询能否顺序访问相邻叶节点。
- 更新节点时结构维护是否稳定。

多阶树的目标是让少量键值占用一个 Page，从而使一个节点指向很多下级 Page。分支越多，树越矮，I/O 次数通常越少。

2.1.2 B 树：内部节点也保存数据

B 树的键值和关联数据可以同时出现在内部节点与叶节点。按主键找到一条记录时，可能在某个非叶节点就提前拿到数据，不一定到叶子。

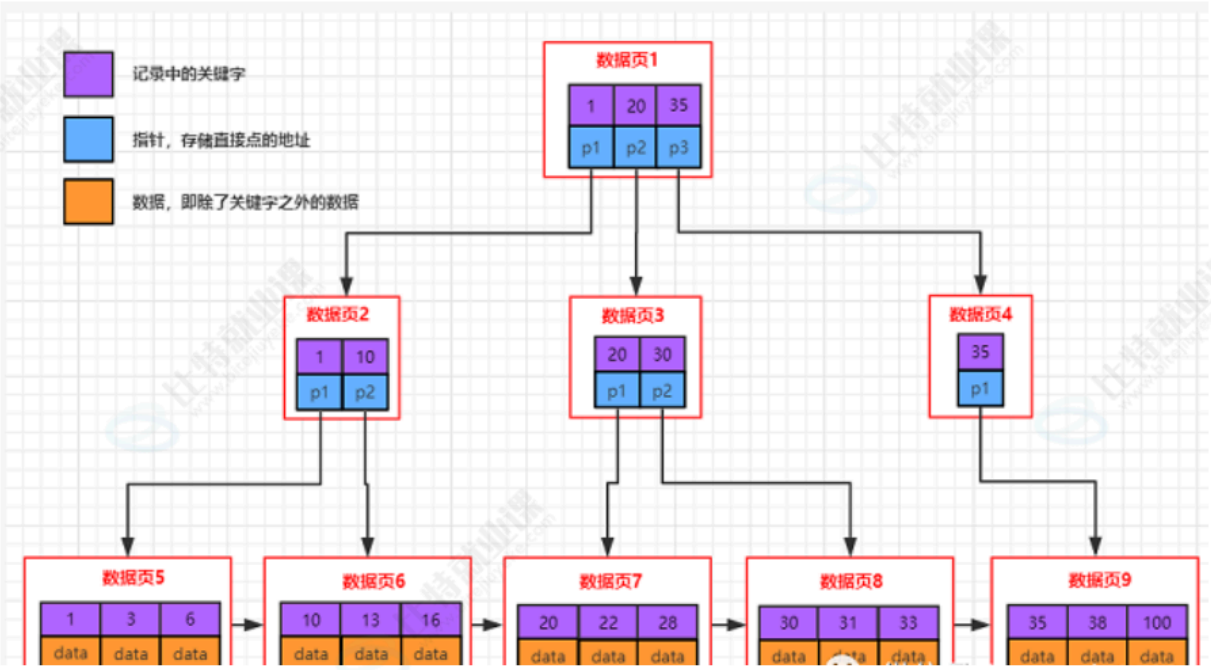


但对数据库索引而言，内部节点保存完整数据会占用本可用来放更多键和指针的空间，导致扇出下降、树高增加。

2.1.3 B+ 树：数据集中在叶节点

B+ 树的内部节点主要保存键值和子 Page 指针，数据集中在叶节点；所有叶节点按顺序相连。

B+树



上面的图，是在网上找的，大家也可以搜一下。

目前这两棵树，对我们最有意义的区别是：

一次点查通常要从根节点逐层找到叶节点；到达目标范围后，再做页内查找。范围查询则可以从起始叶节点开始，沿叶节点链表继续读取，不需要反复回到根节点。

2.1.4 对本节最重要的两点差异

对照项	B 树	B+ 树
数据位置	内部节点和叶节点都可能保存数据	数据集中在叶节点
目录节点可容纳的键	较少	较多
树高	通常更高	通常更低
叶节点连接	没有统一顺序链	全部按顺序连接
点查	可能提前在内部节点命中	通常定位到叶节点
范围查询	需要多次树内定位	找到起点后沿叶链扫描

把选择 B+ 树的原因概括为：

1. 内部节点不保存完整数据，因此能放更多键和 Page 指针，树的扇出更大、整体更矮，I/O 次数更少。
2. 叶节点全部相连，更适合范围查询和顺序扫描。

2.2 为什么其他结构不适合作为 InnoDB 的主索引

2.2.1 链表、二叉搜索树和 AVL/红黑树

- **链表**：查找通常要逐条遍历，复杂度退化为 `O(n)`，并且会产生大量 Page 访问。
- **普通二叉搜索树**：有序插入时可能退化成链表，查询从 `O(log n)` 退化为 `O(n)`。
- **AVL/红黑树**：虽然能维持平衡，但每个节点最多只有两个子节点，树高相对于多阶 B+ 树过高。

数据库的一层节点本质上是一个 16KB Page。二叉树的低扇出意味着同样数量的记录需要更多层 Page；B+ 树则可以用一个 Page 容纳大量键值和子指针。

2.2.2 哈希索引

哈希索引按哈希值定位，等值查询通常接近 `O(1)`，但不适合范围查询、排序和有序扫描，因为哈希值的大小顺序与原始键值顺序无关。

课件表中常见索引类型如下：

存储引擎	可使用的索引类型
InnoDB	BTREE
MyISAM	BTREE
MEMORY/HEAP	HASH、BTREE

> 这里的 `BTREE` 是 MySQL 对 B 树家族索引类型的标识，并不表示底层使用“二叉搜索树”。

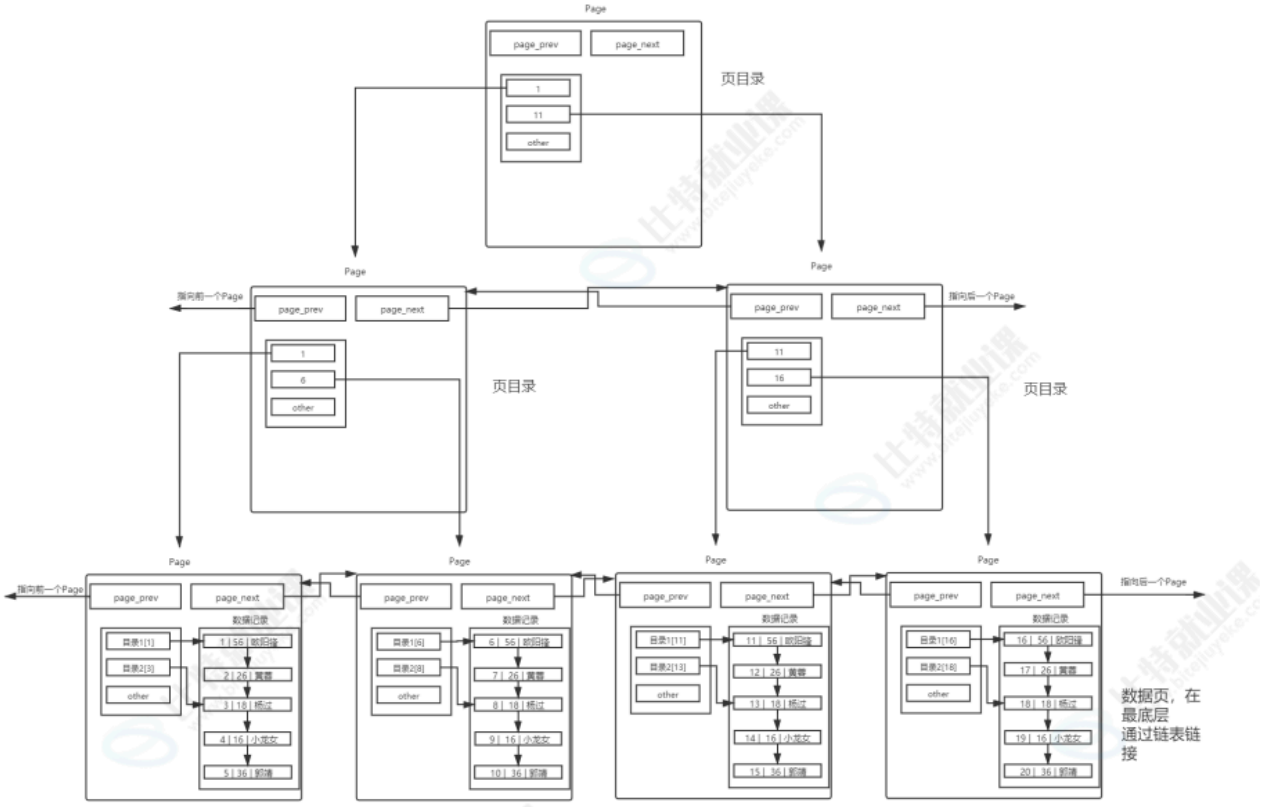
2.2.3 InnoDB 与 MyISAM 为什么采用 B+ 树

两者都使用 B+ 树组织索引。真正不同的是叶节点“指向哪里”：

- InnoDB 聚簇索引的叶节点直接保存整行数据。
- MyISAM 索引的叶节点保存数据记录所在位置。
- InnoDB 辅助索引的叶节点保存该记录的主键值。

2.3 从页目录理解树高

2.3.1 每一层目录都在缩小范围



这货就是传说中的B+树啊！没错，至此，我们已经给我们的表user构建完了主键索引。

随便找一个id=？我们发现，现在查找的Page数一定减少了，也就意味着IO次数减少了，那么效率也就提高了。

图中最上层 Page 保存较小区间的最小键值和下级目录 Page 的地址；下层目录再指向真正的数据 Page。查找时只加载路径上的少量 Page，不必把所有数据 Page 都读入内存。

2.3.2 根到叶的路径就是索引查找路径

可以把 B+ 树查找概括为：

```
根页比较键值
    ↓
选择可能包含目标键的子页
    ↓
重复目录页比较
    ↓
到达叶页
    ↓
页内目录 + 记录定位
```

树高越低，这条路径越短；叶页容量越大，树能覆盖的数据就越多。B+ 树正是为了提高这个工程指标。

3. InnoDB 聚簇索引

3.1 索引与数据保存在同一棵主键树

3.1.1 聚簇索引的含义

InnoDB 使用主键 B+ 树组织整张表，叶节点保存完整行记录。这种“表数据与主键索引放在一起”的结构称为聚簇索引（Clustered Index）。

```
CREATE TABLE test1 (
    id INT PRIMARY KEY,
    name VARCHAR(20) NOT NULL
) ENGINE=InnoDB;
```

在上述表中，`id` 既定义唯一主键，也成为 InnoDB 组织表数据的键。按主键查找时，从主键 B+ 树根页逐层定位到叶页，叶页中的目标记录就是完整数据。

3.1.2 为什么主键设计很重要

主键不仅是一项约束，还决定聚簇索引的键：

- 主键不宜过长，否则辅助索引保存主键值时会放大空间成本。
- 主键不宜频繁变化，否则聚簇索引和所有辅助索引中的引用都可能需要调整。
- 顺序性较好的主键通常有利于降低随机写入和页分裂压力。
- 没有显式主键时，InnoDB 会选择其他合适候选或内部行标识，业务代码不应依赖这个选择。

3.1.3 从表文件观察聚簇索引

课件使用旧版 MySQL 目录观察 InnoDB 表：

1. `test1.frm`：表结构。
2. `test1.ibd`：表数据与索引等数据。

```
total 112
-rw-r-----. 1 mysql mysql    61 Oct 26 01:19 db.opt
-rw-r-----. 1 mysql mysql  8586 Oct 26 01:20 test1.frm
-rw-r-----. 1 mysql mysql 98304 Oct 26 01:20 test1.ibd
```

> 原速记中的 `test1.idb` 应更正为 `test1.ibd`，即 InnoDB 表空间文件。MySQL 8.0 默认不再为每张表单独保留 `.frm` 来保存表定义，目录结构要以当前 MySQL 版本为准。

3.2 InnoDB 辅助索引与回表

3.2.1 辅助索引叶节点保存主键

对非主键列建立普通索引后，InnoDB 会维护另一棵 B+ 树。这棵树的叶节点一般保存：

```
辅助索引列的值 + 对应记录的主键值
```

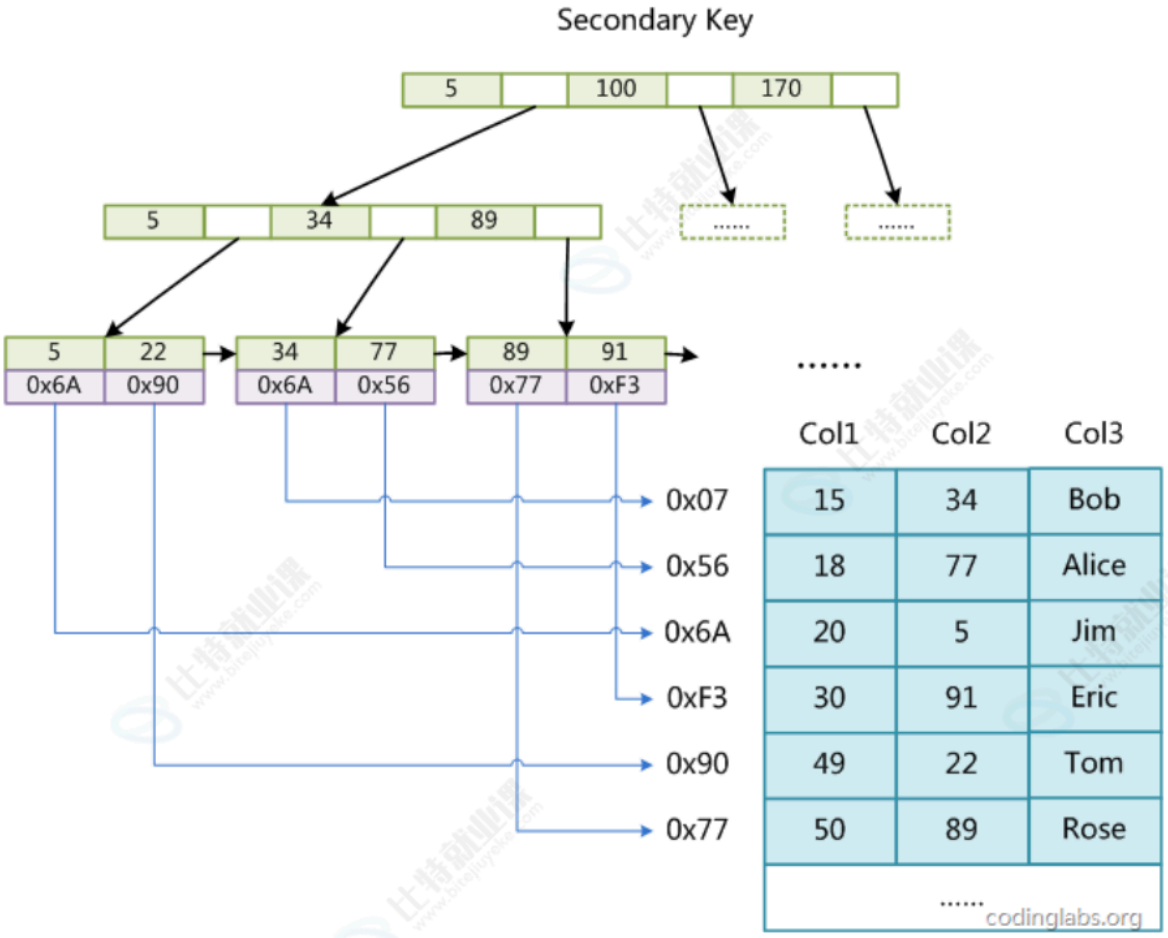
叶节点不重复保存整行数据，以减少每个索引页的体积、提高每个 Page 能容纳的索引项数量。

3.2.2 回表查询的两步路径

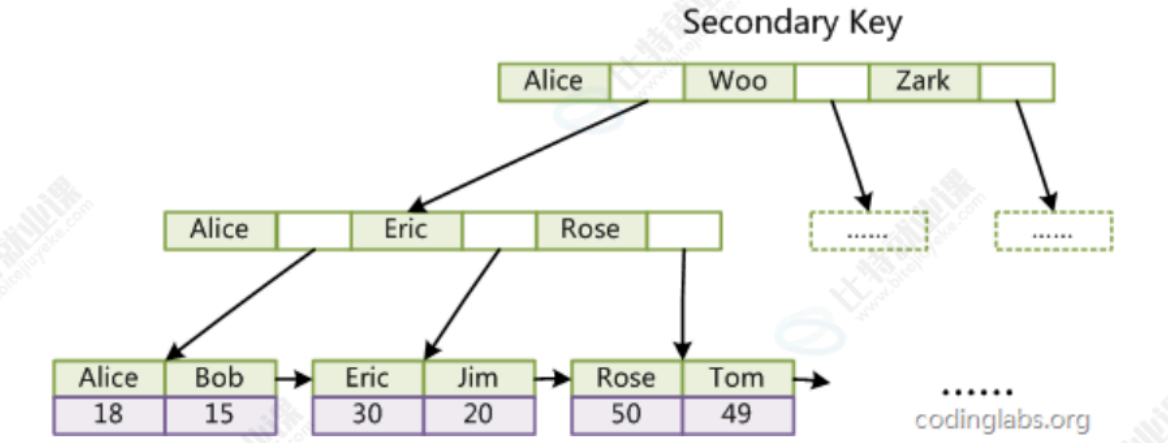
假设查询 `name` 并要返回整行数据：

1. 先走 `name` 辅助索引，找到对应主键 `id`。
2. 再用 `id` 走主键聚簇索引，读取整行记录。

第二次按主键查聚簇索引的过程叫“回表”。如果目标行很多、结果有大量随机主键，回表会产生更多 Page 访问。



同样，`InnoDB` 除了主键索引，用户也会建立辅助（普通）索引，我们以上表中的 `Col3` 建立对应的辅助索引如下图：



可以看到，`InnoDB` 的非主键索引中叶子节点并没有数据，而只有对应记录的key值。

所以通过辅助（普通）索引，找到目标记录，需要两遍索引：首先检索辅助索引获得主键，然后用主键

3.2.3 覆盖索引为什么能减少回表

如果查询需要的列已经全部包含在同一个辅助索引中，例如索引为 `(name, age)` 且查询只取 `name, age`，数据库可以直接从辅助索引叶节点得到结果，不必再按主键回表。

这并不是新增一种索引类型，而是“当前查询恰好被某个索引覆盖”。实际判断应结合 `EXPLAIN` 的 `key`、`Extra` 等信息。

3.3 一张表为什么对应多棵树

3.3.1 主键树和辅助索引树同时存在

InnoDB 表中：

- 一张表只有一个聚簇索引，通常由主键组织。
- 每建立一个辅助索引，就多维护一棵独立的 B+ 树。
- 主键树叶子保存整行；辅助索引树叶子保存索引列和主键。

因此“索引越多，查询越快”并不成立。每多一棵索引树，就多占磁盘和 Buffer Pool，也多一份增删改维护工作。

3.3.2 更新辅助索引列的代价

更新某行数据时：

- 聚簇索引中的行可能需要调整。
- 被修改列涉及的每个辅助索引都可能需要删除旧键、插入新键。
- 索引 Page 可能分裂、合并，或需要写回磁盘。

所以写多读少的表尤其要控制索引数量。

4. MyISAM 非聚簇索引

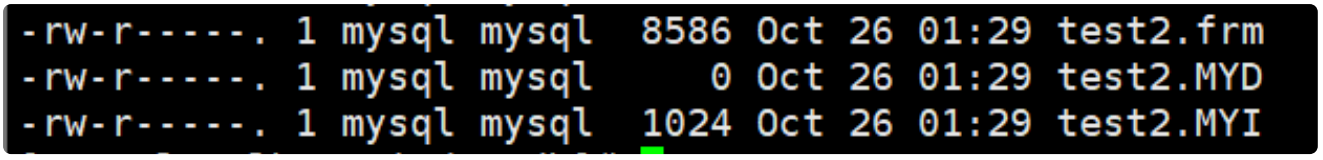
4.1 数据与索引分文件保存

4.1.1 MyISAM 的典型文件

```
CREATE TABLE test2 (
    id INT PRIMARY KEY,
    name VARCHAR(20) NOT NULL
) ENGINE=MyISAM;
```

课件环境中可以看到：

文件	作用
`.frm`	表结构
`.MYD`	MyISAM Data，表数据
`.MYI`	MyISAM Index，索引数据



这体现的是旧版 MySQL 的文件布局；新版本的元数据管理方式可能不同，但 `MYD` 数据文件与 `MYI` 索引文件分离这一核心特征仍可用来理解 MyISAM。

4.1.2 为什么称为非聚簇索引

MyISAM 的索引 Page 和数据 Page 分开保存。B+ 树叶子中保存的不是完整行，而是记录地址，通过地址再去 `.MYD` 读取数据行。

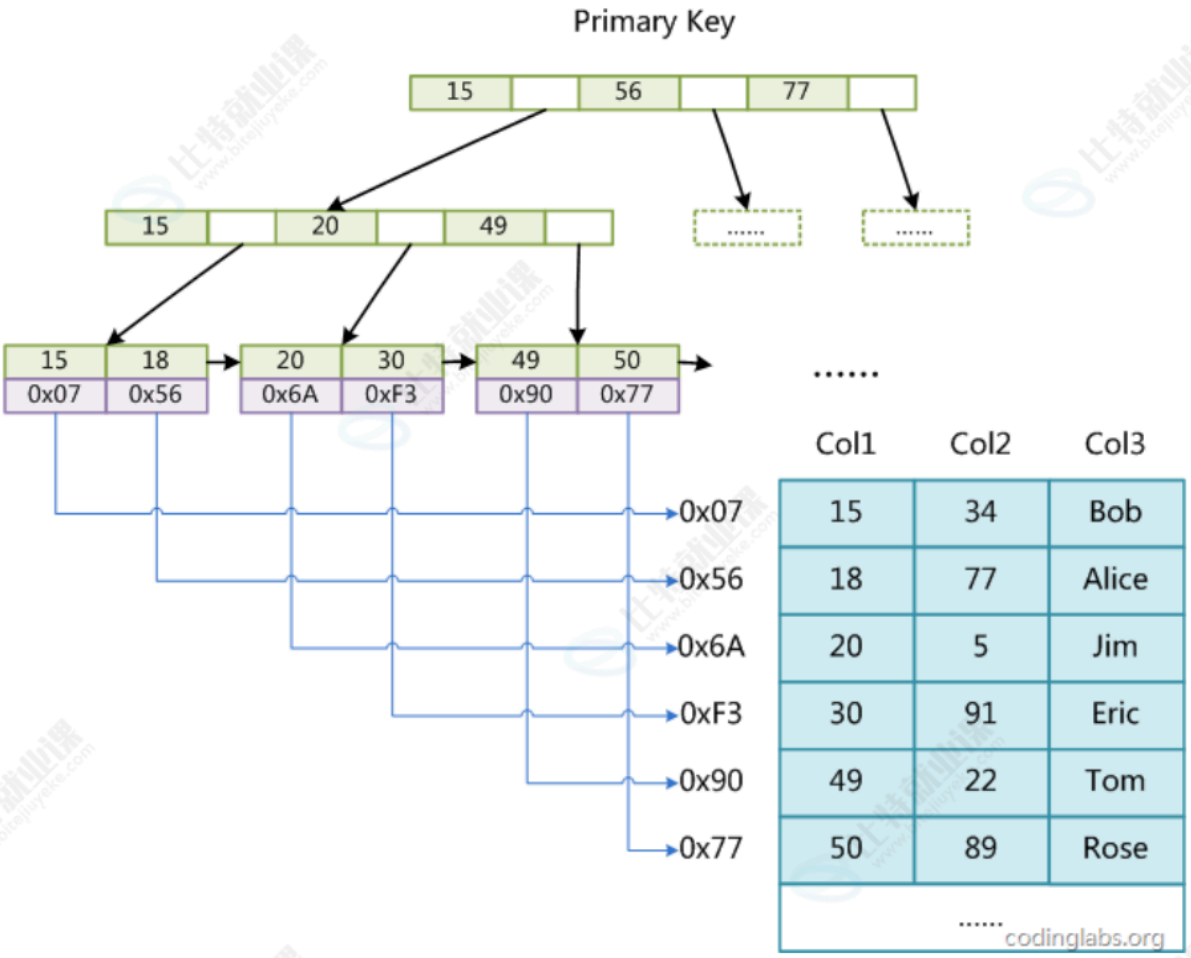
这种“用户数据与索引数据分离”的方案称为非聚簇索引。

4.2 MyISAM 主键索引

4.2.1 叶节点保存行地址

MyISAM 存储引擎-主键索引

MyISAM 引擎同样使用B+树作为索引结果，叶节点的data域存放的是数据记录的地址。下图为 MyISAM 表的主索引，Col1 为主键。



其中，MyISAM 最大的特点是，将索引Page和数据Page分离，也就是叶子节点没有数据，只有对应数据的地址。

相较于 InnoDB 索引，InnoDB 是将索引和数据放在一起的。

知识点可以概括为：

```
MyISAM 主键 B+ 树
   ↓ 叶节点保存数据记录地址
MYD 数据文件中的目标行
```

主键键值决定索引顺序，但完整列数据不在主键索引叶节点中。

4.2.2 从索引到数据通常多一步寻址

InnoDB 主键查询在聚簇索引叶页直接得到整行；MyISAM 主键查询需要：

1. 在 `.MYI` 的主键 B+ 树中定位记录地址。

2. 按地址到 `.MYD` 中读取数据。

4.3 MyISAM 辅助索引

4.3.1 与主键索引结构相同

MyISAM 对普通列建立辅助索引时，B+ 树的叶节点同样保存数据记录地址。主键索引与辅助索引的主要差别不是叶节点定位方式，而是：

- 主键值唯一且非空。
- 普通索引列允许重复。

因此 MyISAM 的普通索引能够直接通过记录地址找到数据行，不需要像 InnoDB 辅助索引那样先拿到主键再回表。

4.3.2 InnoDB 与 MyISAM 对照

对照项	InnoDB	MyISAM
主索引结构	B+ 树聚簇索引	B+ 树非聚簇索引
主键叶节点内容	完整行记录	数据记录地址
辅助索引叶节点内容	辅助索引列 + 主键值	数据记录地址
存取分离	主数据与索引在同一表空间结构中	`MYD` 数据和 `MYI` 索引分离
辅助索引查完整行	可能需要回表	按叶节点地址读取数据

5. 索引操作

5.1 创建主键索引

5.1.1 在字段定义处声明

```
CREATE TABLE user1 (
    id INT PRIMARY KEY,
    name VARCHAR(30)
);
```

这是最直接的方式，声明主键的同时由 MySQL 创建主键索引。

5.1.2 在表定义末尾声明

```
CREATE TABLE user2 (
    id INT,
    name VARCHAR(30),
    PRIMARY KEY (id)
);
```

如果主键由多列共同组成，可以在表级约束中写成 `PRIMARY KEY (col1, col2)`。

5.1.3 建表后再添加

```
CREATE TABLE user3 (
    id INT,
    name VARCHAR(30)
);

ALTER TABLE user3
ADD PRIMARY KEY (id);
```

主键索引的特点：

- 一个表只能有一个主键。
- 主键列不能为 `NULL`。
- 主键值不能重复。
- 主键可以是一列，也可以是多列组成的复合主键。
- InnoDB 通常围绕主键组织聚簇索引。

5.2 创建唯一索引

5.2.1 三种常见写法

```
-- 列后直接声明 UNIQUE
CREATE TABLE user4 (
    id INT PRIMARY KEY,
    name VARCHAR(30) UNIQUE
```

```
);

-- 表级唯一约束
CREATE TABLE user5 (
    id INT PRIMARY KEY,
    name VARCHAR(30),
    UNIQUE (name)
);

-- 建表后添加
CREATE TABLE user6 (
    id INT PRIMARY KEY,
    name VARCHAR(30)
);

ALTER TABLE user6
ADD UNIQUE (name);
```

也可以显式命名：

```
CREATE UNIQUE INDEX uk_user6_name
ON user6 (name);
```

5.2.2 唯一索引的特点

- 一个表可以有多个唯一索引。
- 索引列中的非空值不能重复。
- MySQL 中可空唯一列的 `NULL` 语义与普通值不同，不能简单把“可空唯一索引”当作主键。
- 如果唯一索引列同时为 `NOT NULL`，其约束效果接近主键；但是否作为 InnoDB 聚簇索引，仍以表的主键定义为准。

5.3 创建普通索引

5.3.1 建表时声明

```
CREATE TABLE user8 (
    id INT PRIMARY KEY,
    name VARCHAR(20),
    email VARCHAR(30),
```

```
    INDEX (name)
);
```

5.3.2 建表后添加

```
CREATE TABLE user9 (
    id INT PRIMARY KEY,
    name VARCHAR(20),
    email VARCHAR(30)
);

ALTER TABLE user9
ADD INDEX (name);
```

5.3.3 显式指定索引名

```
CREATE TABLE user10 (
    id INT PRIMARY KEY,
    name VARCHAR(20),
    email VARCHAR(30)
);

CREATE INDEX idx_user10_name
ON user10 (name);
```

普通索引的特点：

- 一个表可以有多个普通索引。
- 不要求索引列唯一。
- 实际业务中通常应显式命名，例如 `idx_<表>_<列>`，便于查看、删除和维护。
- 对多列建立复合索引时列顺序很重要，后续应继续学习最左匹配原则。

5.4 创建全文索引

5.4.1 示例表

```
CREATE TABLE articles (
    id INT UNSIGNED AUTO_INCREMENT NOT NULL PRIMARY KEY,
```

```
    title VARCHAR(200),
    body TEXT,
    FULLTEXT KEY ft_articles_title_body (title, body)
) ENGINE=InnoDB;
```

> 环境要求全文索引使用 MyISAM，并说明默认分词主要面向英文。现代 MySQL 的 InnoDB 已支持 `FULLTEXT`；中文检索通常还需要按版本选择合适的分词插件，例如 `ngram`。应以部署版本的官方文档为准。

插入示例数据：

```
INSERT INTO articles (title, body) VALUES
    ('MySQL Tutorial', 'DBMS stands for DataBase ...'),
    ('How To Use MySQL Well', 'After you went through a ...'),
    ('Optimizing MySQL', 'In this tutorial we will show ...'),
    ('1001 MySQL Tricks', '1. Never run mysqld as root. 2. ...'),
    ('MySQL vs. YourSQL', 'In the following database comparison ...'),
    ('MySQL Security', 'When configured properly, MySQL ...');
```

5.4.2 普通 `LIKE` 不一定使用全文索引

```
SELECT *
FROM articles
WHERE body LIKE '%database%';
```

虽然能查到数据，但前置通配符通常无法利用普通 B+ 树索引，执行计划可能是全表扫描。

5.4.3 使用 `MATCH ... AGAINST`

```
SELECT *
FROM articles
WHERE MATCH (title, body)
      AGAINST ('database');
```

分析是否使用全文索引：

```
EXPLAIN
SELECT *
```

```
FROM articles
WHERE MATCH (title, body)
      AGAINST ('database');
```

课件结果中可以看到 `type: fulltext`、`possible_keys: title` 和 `key: title`。这说明查询走了全文索引，而不是普通的 `LIKE '%...%'` 扫描。

6. 查看、删除与设计索引

6.1 查看索引

6.1.1 三种常用方式

```
SHOW KEYS FROM goods;
SHOW INDEX FROM goods;
DESC goods;
```

- `SHOW KEYS` 与 `SHOW INDEX` 都能查看索引信息。
- `DESC` 主要查看列与键的简要信息，索引细节较少。

```
mysql> show index from test1\G
*************************** 1. row ***************************
        Table: test1
   Non_unique: 0
     Key_name: PRIMARY
 Seq_in_index: 1
  Column_name: id
    Collation: A
  Cardinality: 0
     Sub_part: NULL
       Packed: NULL
         Null:
   Index_type: BTREE
      Comment:
Index_comment:
1 row in set (0.00 sec)
```

6.1.2 重点字段

字段	含义
`Non_unique`	`0` 表示不能重复，即唯一索引
`Key_name`	索引名，`PRIMARY` 表示主键索引
`Seq_in_index`	该列在复合索引中的顺序
`Column_name`	索引作用列
`Cardinality`	索引基数估计值
`Index_type`	索引结构类型，例如 `BTREE`

`Key_name` 是后续删除普通索引、唯一索引时最重要的索引名来源。

6.2 删除索引

6.2.1 删除主键

```
ALTER TABLE user3
DROP PRIMARY KEY;
```

如果主键列带 `AUTO_INCREMENT`，通常需要先处理自增属性或建立新的主键/索引，具体取决于表结构。

6.2.2 删除普通索引或唯一索引

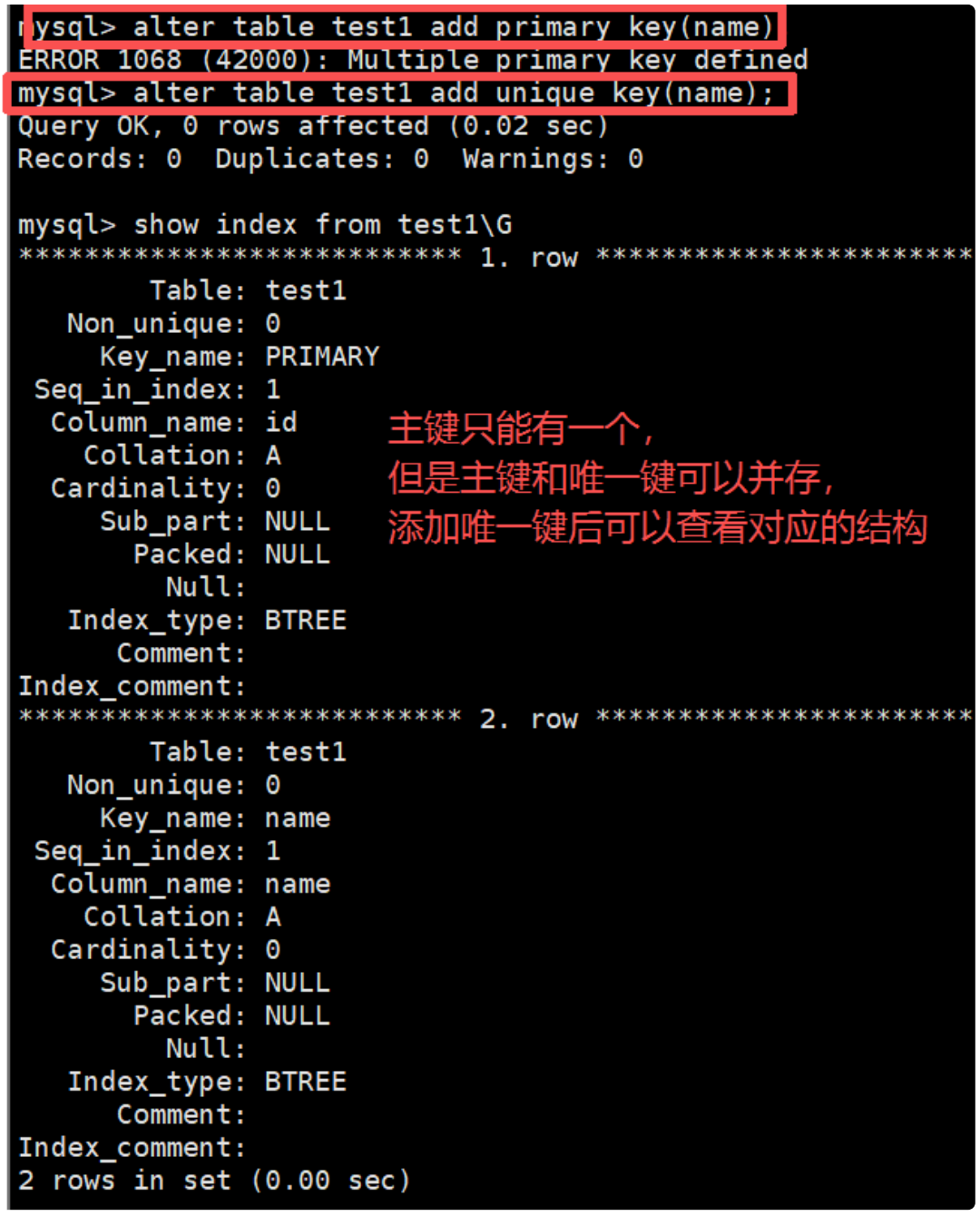
```
ALTER TABLE user10
DROP INDEX idx_user10_name;
```

或：

```
DROP INDEX idx_user10_name
ON user10;
```

删除时应使用 `SHOW INDEX` 中的 `Key_name`，不要凭空拼接 `DROP INDEX unique(name)` 这类不存在的索引名。

6.2.3 主键和唯一键共存时



如果表中同时存在 `PRIMARY` 和唯一索引，`SHOW INDEX` 会显示多行或多个索引项。实际删除命令必须按真实的索引名执行：

```
SHOW INDEX FROM test1;
ALTER TABLE test1 DROP INDEX <SHOW_INDEX_显示的Key_name>;
```

```
ALTER TABLE test1 DROP INDEX unique(name);
```

不是有效的“按列定义删除索引”写法。`UNIQUE` 是建索引时的约束类型；删除时使用的是索引名。

6.3 索引创建原则

6.3.1 适合建索引的场景

- 经常出现在 `WHERE` 条件中的列。
- 经常参与 `JOIN`、`ORDER BY`、`GROUP BY` 的列。
- 数据量大且查询只返回少量目标记录的场景。
- 选择性较高、能把大范围迅速缩小的列。

6.3.2 不适合单独建索引的场景

- 唯一性很差的列，例如只有少数几种取值；单列索引往往不能筛掉足够多的数据。
- 更新非常频繁的列，因为每次更新都可能维护索引。
- 很少出现在查询条件中的列。
- 以大量写操作为主、读取收益不足以抵消维护成本的列。
- 起始通配符的模糊匹配，例如 `LIKE '%keyword%'`，普通 B+ 树通常难以有效利用。

6.3.3 索引设计要结合查询整体判断

是否需要索引，不应只凭“列看起来常查”决定。更可靠的方法是：

1. 收集真实 SQL 和访问频率。
2. 用 `EXPLAIN` 查看访问类型、候选索引和实际索引。
3. 比较增加索引前后的读性能、写入成本和占用空间。
4. 定期清理长期未使用或职责重复的索引。

6.4 复合索引、最左匹配与覆盖索引

6.4.1 复合索引

多个列可以共同组成一个索引：

```
CREATE INDEX idx_user9_name_email
ON user9 (name, email);
```

复合索引不是“分别给两列建两个索引”，而是一棵以列顺序为组织依据的联合 B+ 树。

6.4.2 最左匹配原则

对 `(name, email)` 索引，查询通常可以从最左列开始利用索引；若条件跳开最左列，能否利用索引取决于优化器与后续列条件。

最安全的理解是：

```
索引顺序是 (name, email)
   ↓
先按 name 排序，再在同一 name 内按 email 排序
```

因此复合索引的列顺序必须围绕实际查询模式设计。

6.4.3 覆盖索引

当查询所需列都能从索引中取得，数据库可以避免回表。对 InnoDB 来说，辅助索引叶节点天然包含主键；如果还需要其他列，就要结合索引定义判断是否能覆盖。

7. 易错点与修正

7.1 `Index_type: BTREE` 不是二叉搜索树

`SHOW INDEX` 中的 `BTREE` 表示使用的是 B 树家族索引结构，InnoDB 实际采用 B+ 树组织，不能理解为“二叉树索引”。

7.2 MyISAM 与 InnoDB 的“索引和数据”关系

- InnoDB 主键聚簇索引叶节点保存完整行。
- InnoDB 辅助索引叶节点保存辅助索引列和主键值。
- MyISAM 索引叶节点保存数据记录地址。
- MyISAM 的 `MYD` 与 `MYI` 是分开的文件。

记住“叶节点里究竟放整行、主键还是行地址”，比死记术语更可靠。

7.3 文件名与版本问题

原速记中的 `test1.idb` 应为 `test1.ibd`。课件展示的是旧版 MySQL 的 `.frm`、`.MYD`、`.MYI` 目录；新版本的表结构元数据管理和表空间文件布局可能不同，但索引与数据分离/聚合的核心原理仍在。

7.4 全文索引的版本差异

课件写作时强调 MyISAM 和英文分词，这是当时环境的教学限制。现代 MySQL 中 InnoDB 也支持全文索引；中文检索是否可用以及效果好坏，还取决于版本、分词器和配置。

7.5 唯一索引不等于主键

唯一索引负责“非空值不重复”，一个表可以建立多个唯一索引；主键还要求非空，并且一个表最多只有一个主键。InnoDB 的聚簇组织通常围绕主键展开。

8. 总结与复习

8.1 一张图串起索引“下”部分

```
页内目录与页目录
   ↓
多阶 B+ 树
   ↓
InnoDB 主键聚簇索引：叶节点保存整行
   ↓
辅助索引：叶节点保存主键
   ↓
普通索引查完整行可能需要回表
   ↓
MyISAM：叶节点保存行地址，MYD/MYI 分离
```

8.2 关键结论

8.2.1 B+ 树为磁盘访问而优化

内部目录页只保存键和指针，提高扇出并降低树高；叶子页有序相连，兼顾点查和范围查询。

8.2.2 InnoDB 主键决定聚簇索引

主键不仅保证唯一性，还决定表数据的组织键。主键过长、频繁变化或随机性过强，都可能带来额外成本。

8.2.3 InnoDB 辅助索引通常需要回表

辅助索引叶节点先给出主键，再通过聚簇索引读取整行。覆盖索引可以让部分查询免于回表。

8.2.4 MyISAM 是非聚簇索引

数据在 `.MYD`，索引在 `.MYI`，索引叶节点保存数据记录地址。

8.2.5 索引是一笔读写权衡

索引减少目标查找成本，却增加空间占用、Buffer Pool 压力和写入维护成本。正确做法是基于真实查询和执行计划做取舍。

8.3 自测题

8.3.1 为什么 B+ 树比二叉搜索树更适合磁盘索引?

因为每个 16KB Page 可以容纳较多键和子指针，B+ 树扇出大、树高低，查找过程中的 Page I/O 更少。

8.3.2 B+ 树为什么比 B 树更适合范围查询?

B+ 树的数据集中在叶节点且叶节点顺序相连，找到范围起点后可以沿叶链继续读取。

8.3.3 InnoDB 辅助索引为什么要回表?

辅助索引叶节点通常只保存辅助列和主键；若查询还需要其他列，必须用主键到聚簇索引中读取整行。

8.3.4 MyISAM 主键索引为什么不是聚簇索引?

MyISAM 的索引与数据分别保存在 `.MYI`、`.MYD` 文件中，索引叶节点保存记录地址，完整数据不在索引叶节点中。

8.3.5 为什么 `SHOW INDEX` 比只执行 `DESC` 更适合检查索引?

`SHOW INDEX` 会给出索引名、是否唯一、列顺序、基数、索引类型等详细信息，便于确认结构并执行删除操作。